

MSE 413 Lab 2 Report

Binary Class and Multi Class Classification

Name: Tom Dang
Student ID: 301548582

Name: Sam Drubetsky
Student ID: 301452017

Date: 2025/02/20

 MSE413_assignment2

Background

Classification is the process of organizing ideas and objects into preset categories. Classification algorithms focus on assigning unseen data into distinct categories. They employ labelled training datasets to classify unseen datasets by calculating the probability that unseen data will be transferred to preset categories.

There are 2 types of classification:

- Binary class classification: categorize items by comparing one category over another category
- Multi-class classification: categorize items by comparing one category versus the rest of the data pool

Binary class classification

Train a random forest classifier on the train set with a number of estimators of 20 and a maximum depth of 20:

Prior to training and models it was necessary to first configure the datasets to meet specifications. By first replacing all 41951 NA values with either 0 or the mean value of the column, then removing all columns that yield the same values on repeat(112 columns for dataset b and 116 columns for dataset e) and finally removing all columns that have a correlation higher than .8 (262 columns for dataset c and 240 for dataset f).

Evaluate the performance of the models for cases a to f on the test set by providing the confusion matrix and metrics of classification (accuracy, sensitivity, specificity, and f1 score):

After configuring the datasets and training the models the following data was obtained:

Results for Model a: Confusion Matrix: [[289 1] [23 1]] Accuracy: 0.9235668790 Sensitivity: 0.0416666667 Specificity: 0.9965517241 F1 Score: 0.0769230769 ----- Results for Model b: Confusion Matrix: [[288 2] [24 0]] Accuracy: 0.9171974522	Results for Model d: Confusion Matrix: [[290 0] [24 0]] Accuracy: 0.9235668790 Sensitivity: 0.0000000000 Specificity: 1.0000000000 F1 Score: 0.0000000000 ----- Results for Model e: Confusion Matrix: [[290 0] [24 0]] Accuracy: 0.9235668790
---	---

Sensitivity: 0.0000000000 Specificity: 0.9931034483 F1 Score: 0.0000000000 ----- Results for Model c: Confusion Matrix: [[289 1] [24 0]] Accuracy: 0.9203821656 Sensitivity: 0.0000000000 Specificity: 0.9965517241 F1 Score: 0.0000000000	Sensitivity: 0.0000000000 Specificity: 1.0000000000 F1 Score: 0.0000000000 ----- Results for Model f: Confusion Matrix: [[289 1] [24 0]] Accuracy: 0.9203821656 Sensitivity: 0.0000000000 Specificity: 0.9965517241 F1 Score: 0.0000000000
--	--

Figure: Gini Importance or Mean Decrease in Impurity (MDI)

A confusion matrix is a matrix that compares the predicted passes and fails to how many of those values were guessed correctly or incorrectly meaning that model b performed the worst and model d and e performed flawlessly. Accuracy represents the confusion matrix in a single value: the proportion of correctly classified guesses out of the total dataset. Meanwhile Specificity tracks how many actual negatives the model predicts correctly ~ a specificity of 1 means that all negative values are classified as such, regardless of how many false negatives there may be. While sensitivity measures how many actual positives are correctly classified. Whilst F1 score measures the balance of the model ~ compares the precision and accuracy of the model.

Compare each case's AUC and ROC performance and discuss the results:

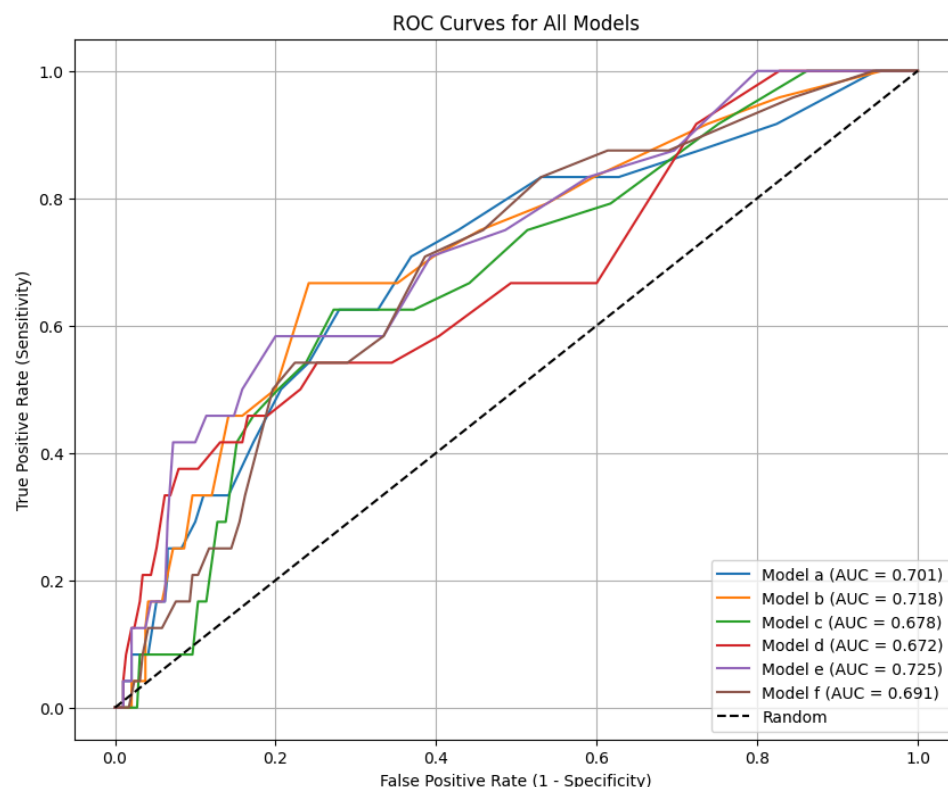


Figure: ROC Graph Comparison of Models a-f

AUC functions as a metric to classify how well a model is performing, if it performs well the AUC value should be higher whilst having an AUC value around 0.5 would represent a coin flip instead of a reasonable prediction.

Between all the models generated in this lab the best one is model e with an AUC of 0.725 and the worst one is model D with an AUC of 0.672.

Gini Importance or Mean Decrease in Impurity (MDI) calculates each feature importance as the sum of the number of splits (across all trees) that include the feature, proportional to the number of samples it splits. Using Mean Decrease in Impurity, show the top 20 features that have the highest importance in prediction for cases a to f and discuss the results:

There are quite a few common features across models such as: feature 51 in Models a and d, Feature 34 in Models a, d, and e, and Feature 32 in Models a and d. These features are variables that influence predictions, regardless of dataset variations or modeling constraints. Meanwhile some features are important in one model but absent in others, meaning they are important on a case-specific basis such as Feature 173 to Model c and Feature 562 is exclusive to Model f both ranking quite high in their feature list.

The top feature in each model has an importance score of approximately 0.021 while the 20th feature typically has a score close to 0.009. The small gap between the top and bottom of the top 20 features implies that multiple features contribute meaningfully to predictions, rather than a model that is largely dependent on a single variable.

Model A		Model B		Model C	
Feature	Importance	Feature	Importance	Feature	Importance
51	0.021687	53	0.021687	82	0.021687
34	0.019525	35	0.019525	53	0.019525
32	0.017406	33	0.017406	48	0.017406
118	0.015798	121	0.015798	173	0.015798
218	0.014708	246	0.014708	78	0.012346
47	0.012346	48	0.012346	175	0.012258
119	0.012258	122	0.012258	476	0.012072
178	0.012072	185	0.012072	238	0.012033
151	0.012033	156	0.012033	200	0.011649
131	0.011649	134	0.011649	565	0.011341
203	0.011341	217	0.011341	120	0.011162
80	0.011162	82	0.011162	225	0.010987
148	0.010987	153	0.010987	121	0.010375
81	0.010375	83	0.010375	247	0.010334
154	0.010334	159	0.010334	83	0.010021
52	0.010021	54	0.010021	81	0.009742
227	0.009837	255	0.009837	41	0.009671
50	0.009742	51	0.009742	85	0.009523
28	0.009671	29	0.009671	474	0.009457
54	0.009523	56	0.009523	32	0.009188

Figure: Gini Importance or Mean Decrease in Impurity (MDI) Models a-c

Model D		Model E		Model F	
Feature	Importance	Feature	Importance	Feature	Importance
51	0.021687	56	0.021687	64	0.021687
34	0.019525	36	0.019525	40	0.019525
32	0.017406	34	0.017406	38	0.017406
118	0.015798	125	0.015798	153	0.015798
218	0.014708	250	0.014708	562	0.014708
47	0.012346	51	0.012346	59	0.012346
119	0.012258	126	0.012258	156	0.012258
178	0.012072	196	0.012072	268	0.012072
151	0.012033	160	0.012033	211	0.012033
131	0.011649	138	0.011649	170	0.011649
203	0.011341	221	0.011341	500	0.011341
80	0.011162	86	0.011162	102	0.011162
148	0.010987	157	0.010987	201	0.010987
81	0.010375	87	0.010375	103	0.010375
154	0.010334	163	0.010334	214	0.010334
52	0.010021	57	0.010021	67	0.010021
227	0.009837	270	0.009837	579	0.009837
50	0.009742	55	0.009742	63	0.009742
28	0.009671	30	0.009671	33	0.009671
54	0.009523	59	0.009523	71	0.009523

Figure: Gini Importance or Mean Decrease in Impurity (MDI) Models e-g

Pseudo Code Completion

```
# Load dataset
try load 'uci-secom.csv'
except FileNotFoundError: print error

# Drop first column
remove first column from dataset

# Create variations of dataset
secom_df_a = fill missing values with 0
secom_df_b = drop single-value columns from secom_df_a
secom_df_c = drop highly correlated columns (>0.8) from secom_df_b
secom_df_d = fill missing values with mean
secom_df_e = drop single-value columns from secom_df_d
secom_df_f = drop highly correlated columns (>0.8) from secom_df_e

# Train and evaluate models
initialize RandomForestClassifier
datasets = { 'a': secom_df_a, 'b': secom_df_b, 'c': secom_df_c, 'd': secom_df_d, 'e':
secom_df_e, 'f': secom_df_f }
results = {}

for model_name, data in datasets:
    split data into train/test
    train model on train set
    predict on test set
    compute confusion matrix, accuracy, sensitivity, specificity, f1 score
    store results

# Compare model performance
create table of accuracy, sensitivity, specificity, f1 score
print table

# Plot ROC curves and compute AUC scores
for each dataset:
    train model, compute predicted probabilities
    compute ROC curve, AUC score
    store AUC score
    plot ROC curve
print AUC score table, show plot

# Analyze feature importance
importance_results = {}
for model_name in results:
    get feature importances
    sort top 20 features
    print table of top features
```

store sorted importances

extract top features for each model

for each model pair:

find shared features

compare importances, print table

print summary of shared features

Multi-class classification

Show the 10 first rows of the data set. Explain the classification of which robots are more challenging based on the 10 first rows

- Based on the data, robot 2 seems to be hard to classify, since the number and data are more significant than the rest. This problem can be both hard and easy to classify depending on the classification models used.

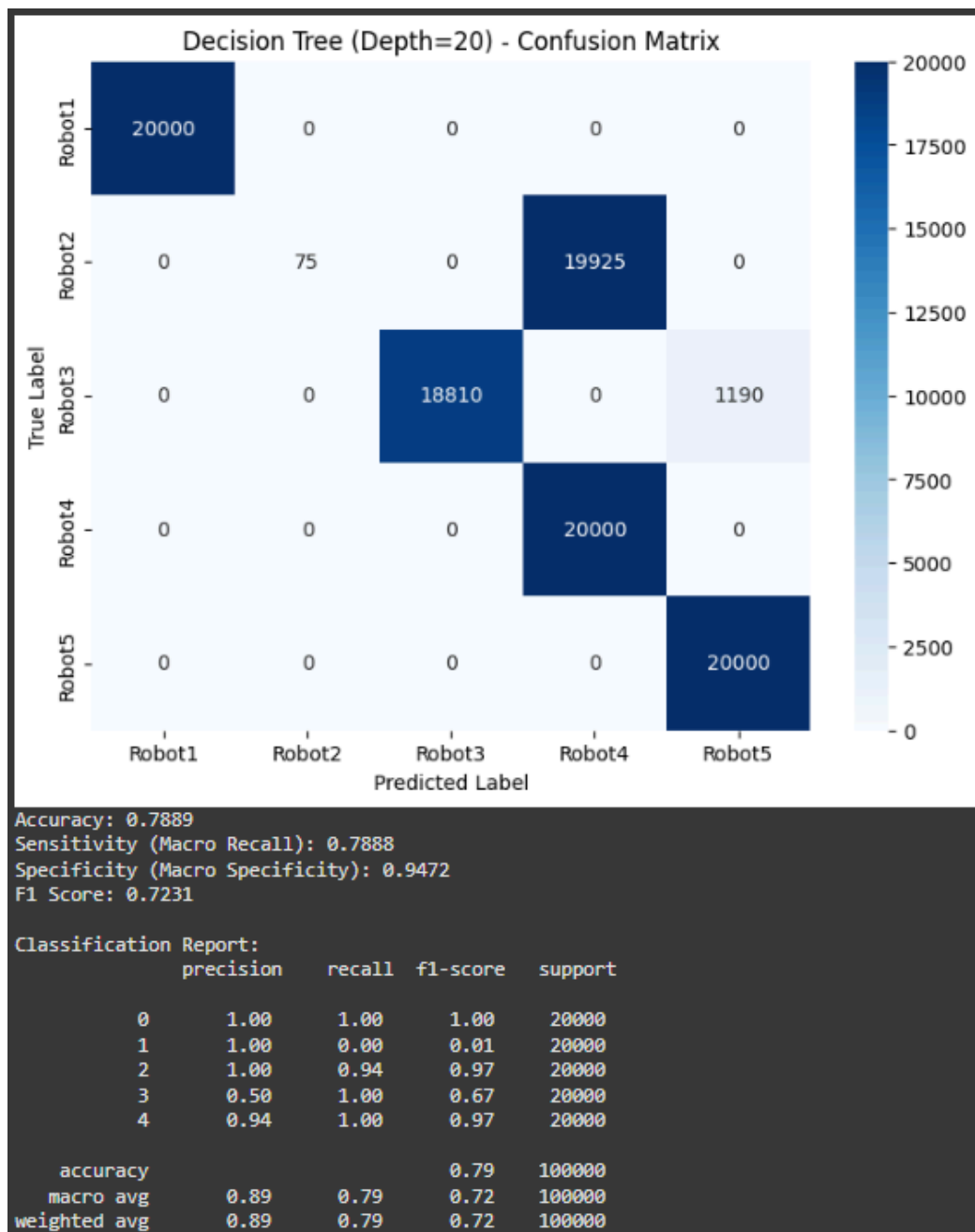


Figure: Decision Tree Classification on the first run

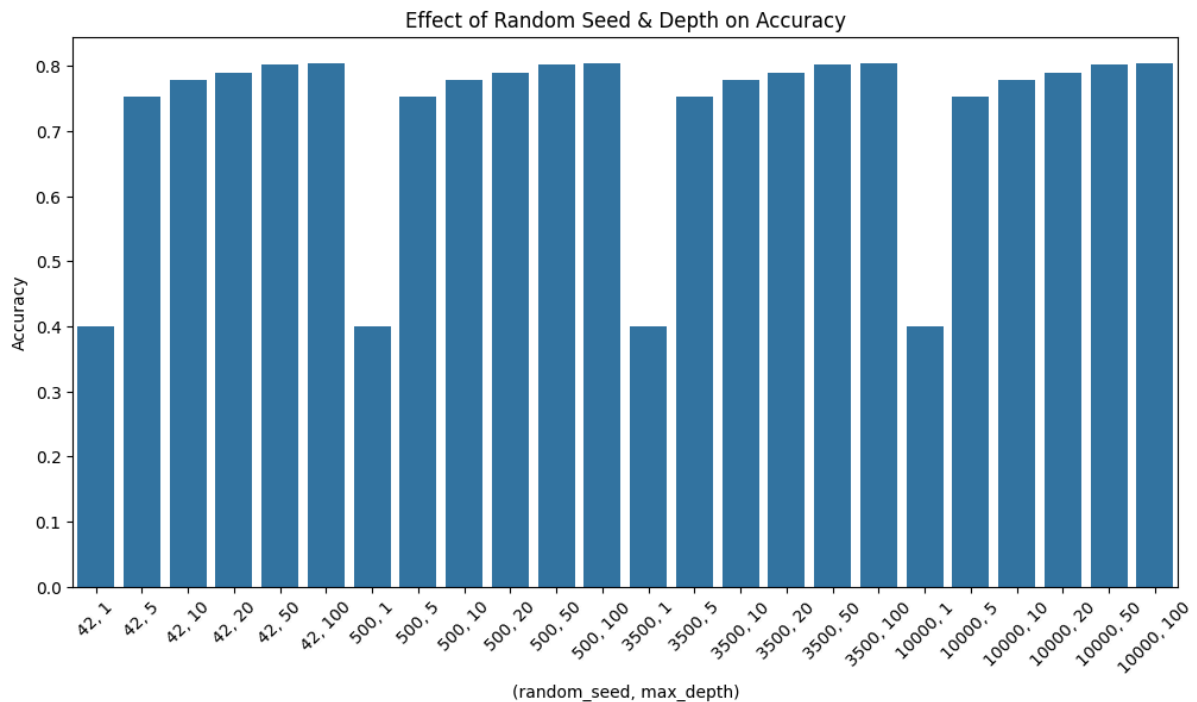


Figure: Performance of Decision Tree with Different Random Seed and Max Depth

The decision tree classifier does well on specificity with about 95% but does in the 70% range on the accuracy, sensitivity and F1 score. In this classification model, the model fails to predict the existence of robot 2 most of the time. Instead, the classifier classifies Robot 2 and Robot 4 at the same rate. Besides that, other robots are classified correctly as intended.

When testing hyper-parameters, the random seed and maximum depth of the tree are 2 important parameters. Random seed keeps the number unbiased toward a specific data cluster. The depth of a tree helps separate classes based on features, but it's also easy to be overfitting if the depth of division is too deep, or underfitting if the depth of division is too shallow. When the deep is 1, there is no division work happening to the classifier, which results in a 50-50 chance of predicting the right category in this case. As the depth goes deeper, the performance of the decision tree classifier improves but at a diminishing rate. In this case, the result is stagnated at about less than 80% accuracy.

Combined with the heat map, the data from Robot 2 distorted the performance of the decision tree model. Even after applying data normalization, the disparity in certain parameters of the data worsens the performance of this model

Overall, the random seed has no effect or marginally positive effect in making the classifier perform better. On the other hand, the maximum depth has a diminishing effect on the performance of the classifier. The decision tree does require normalized data, and the data itself to be balanced or not too separate apart.

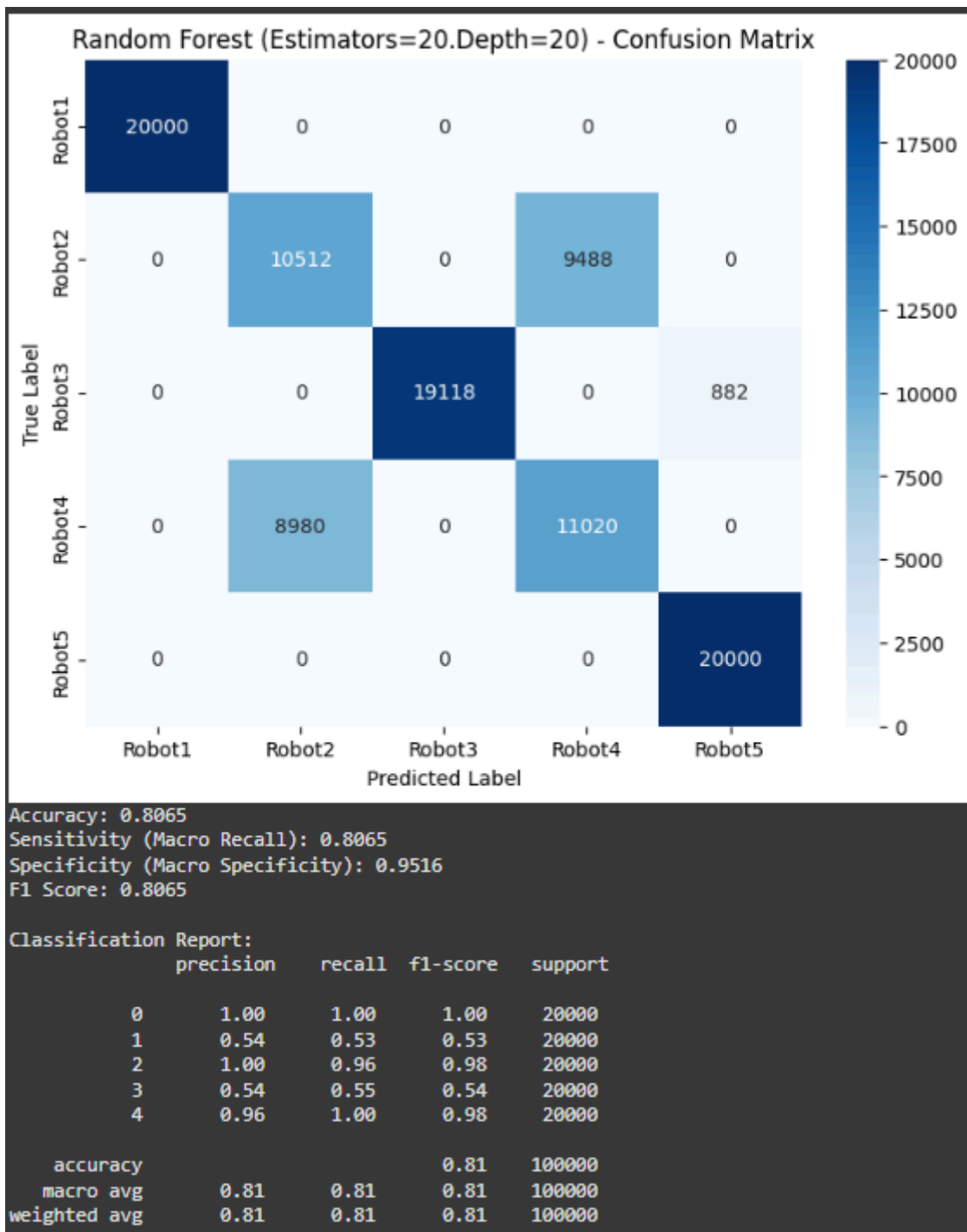


Figure: Random Forest Classification on the first run

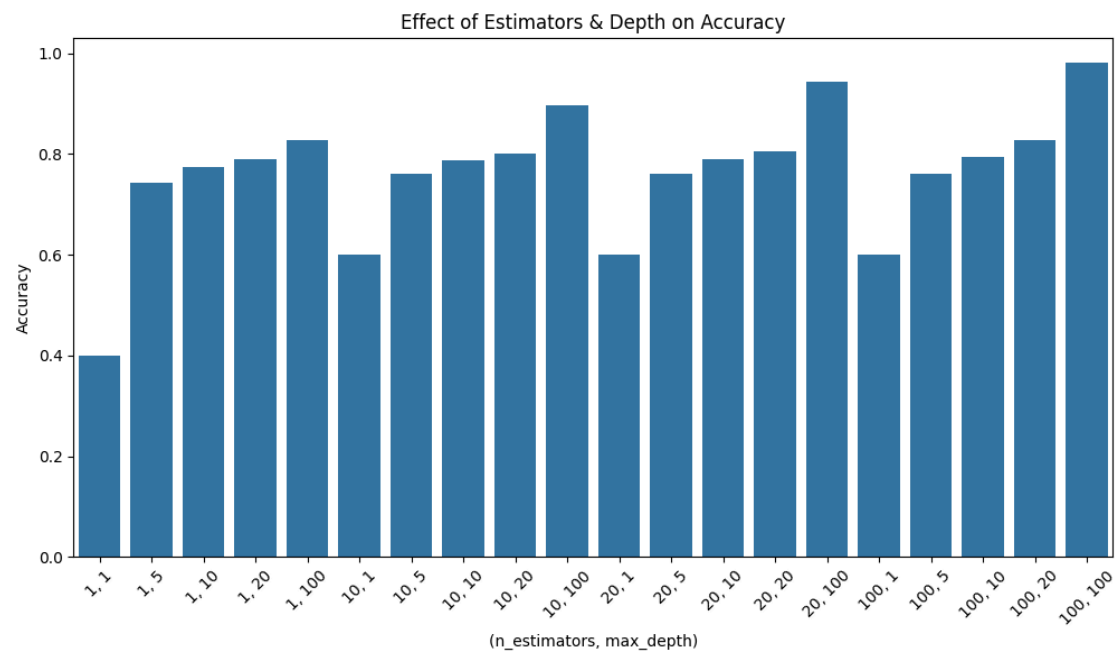


Figure: Performance of Random Forest with Varying Numbers of Trees and Maximum Depth
Run on Google Collab

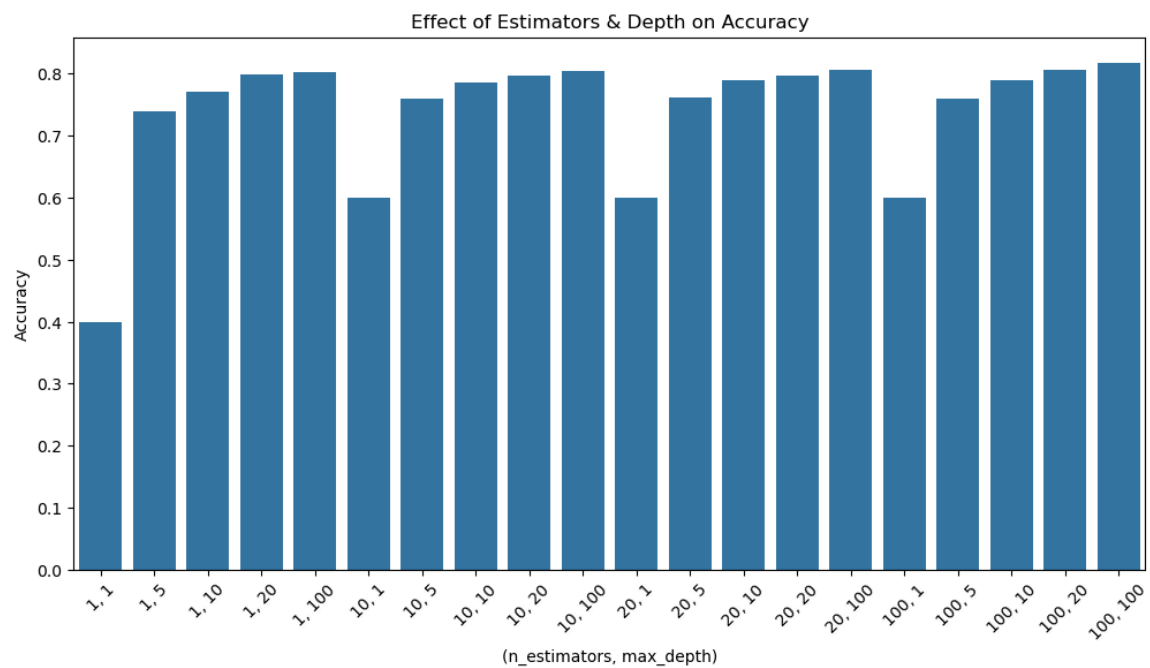


Figure: Performance of Random Forest with Varying Numbers of Trees and Maximum Depth
Run on Jupyter Notebook.

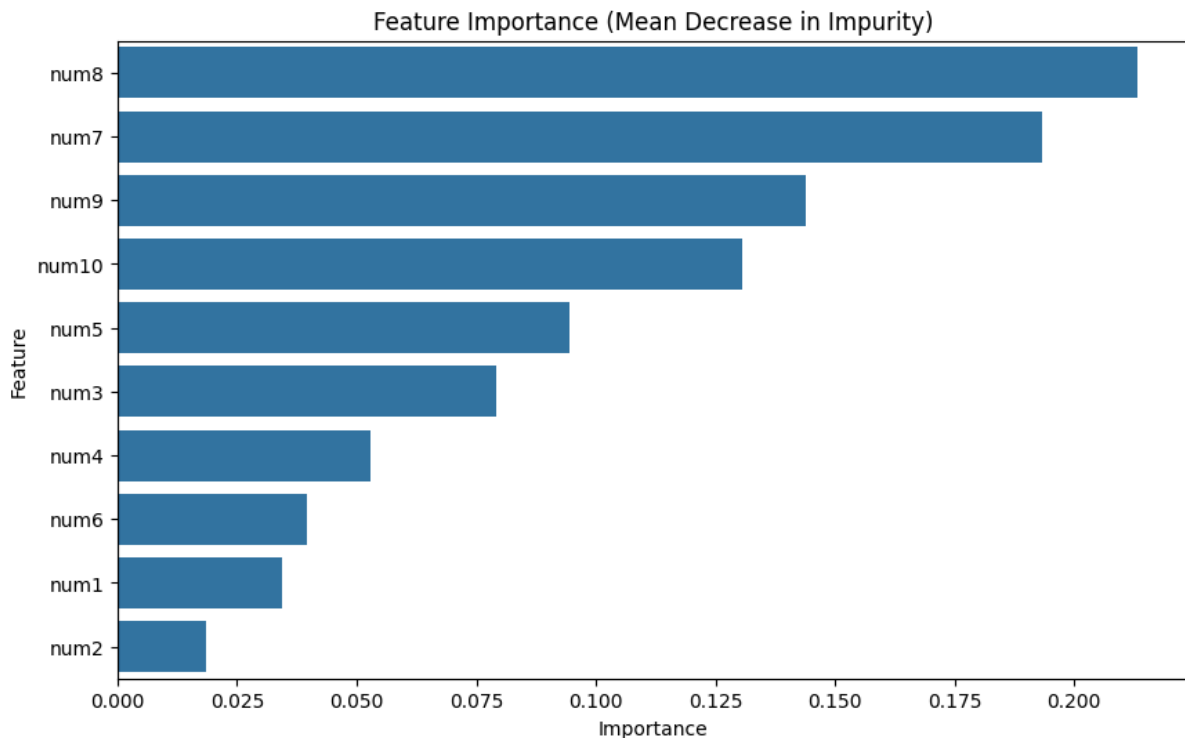


Figure: Feature Importance (Mean Decrease in Impurity)

A single decision tree can not handle predicting the existence of Robot 2 and often mix up between Robot 2 and Robot 4. Multiple decision trees can be handled at a better rate called random forest classification. The model can now better classify the category of Robot2 but still mix up with Robot 4 categories. In the first run, the specificity is as high as a single decision tree. A small improvement on other metrics of classification, such as accuracy, precision and F1 score closer to 80% score.

When testing the hyper-parameters for the random forest classifier, the number of decision trees and the maximum depth of individual trees are 2 most important parameters. The number of decision trees helps the voting process be more precise since the model itself is an ensemble of multiple different smaller components. The depth of each tree improves the specificity of each tree when classifying, as shown in the decision tree model analysis.

For comparison, when the number of estimators is 1, the random forest model performs better compared to the decision tree model. The quantity of estimators also shows a positive effect on the model's performance as it increases. The depth of each decision tree makes the classification more accurate since every tree has an inherently high specificity, which can be beneficial in voting for the final result of the predicted items into a category. Voting among different is shown to be an effective strategy as the accuracy of the model improves with more trees with deep divisions.

Based on the feature importance chart, feature "num 8" is the most critical feature in this data, contributing about 22% of the decision. The top 3 features are "num 8", "num 7", and "num 9". Each is above the estimated 13% of the decision. The bottom 3 features for consideration are "num 2", "num 1", and "num 6", with each less than 5% of the decision

Overall, the random forest model relies on a good balance of the number of estimators and the maximum depth of individual estimators for maximum accuracy. There is a diminishing effect on accuracy when increasing the number of estimators and the maximum depth for the model. Accessing the significance of each feature is also important since it can gain insights about redundant variables and help reduce training time for the model.

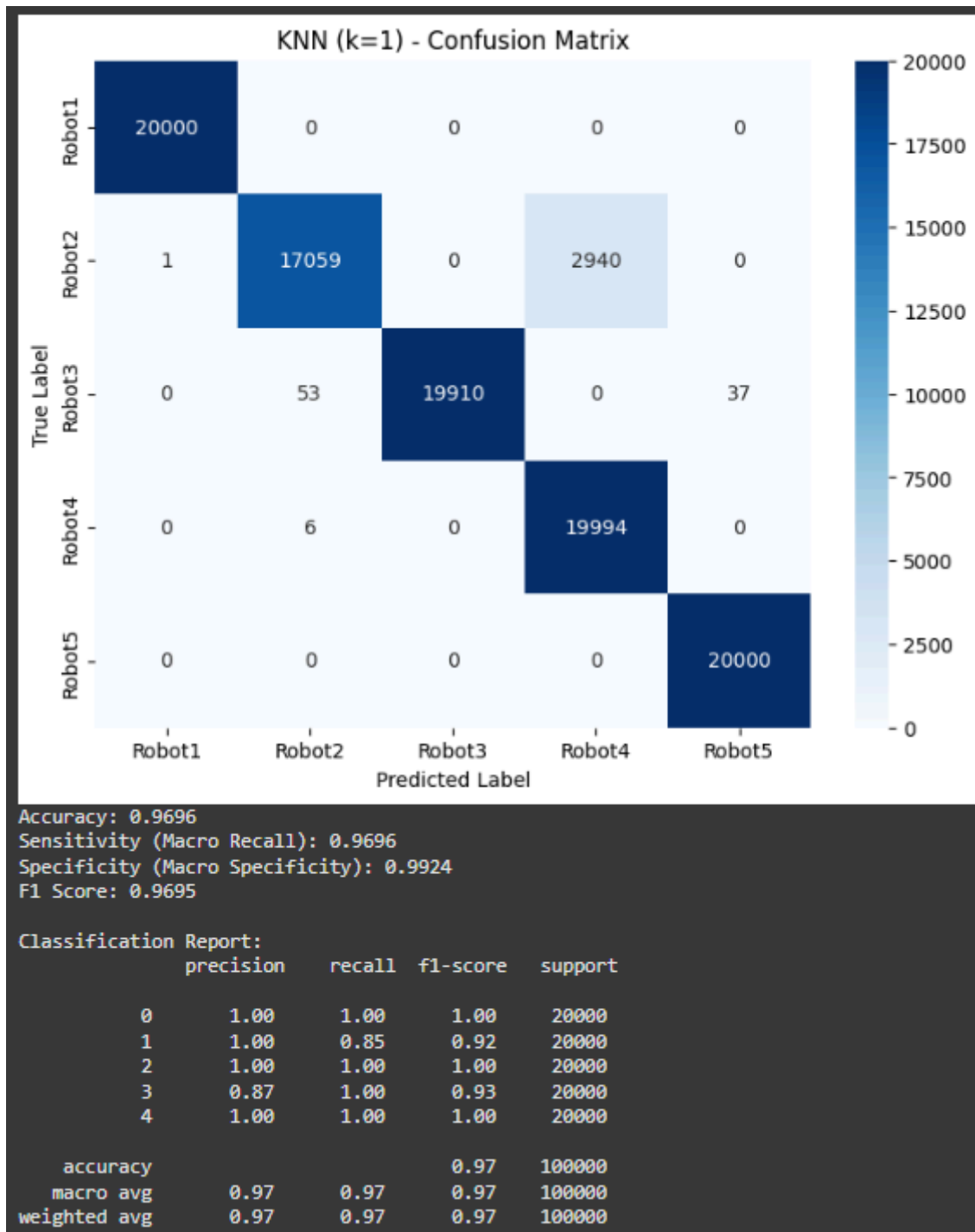


Figure: K Nearest Neighbor (KNN) on the First Run

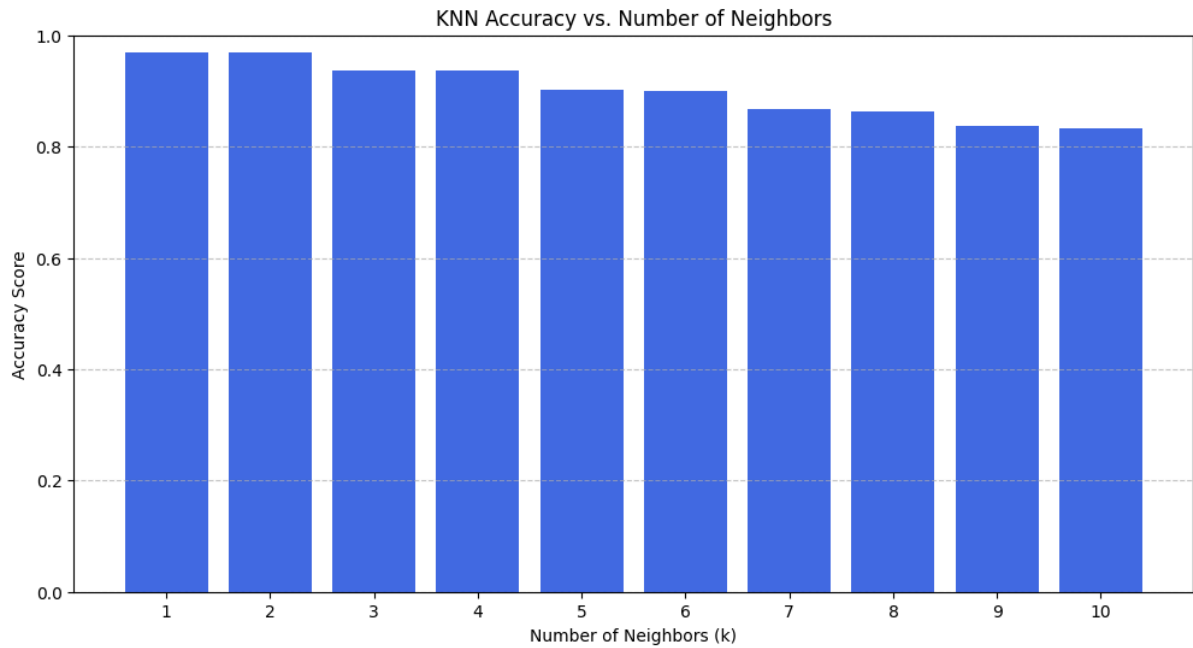


Figure: Performance of KNN Based on the Number of Neighbours

Based on the first, the result provided is much better than the decision tree model and random forest model. There is still an overlap in mixing between Robot 2 and Robot 4 when predicting Robot 4. It archives 96% on the accuracy, precision and F1 score with about 99% specificity score.

When testing the hyper-parameter, the number of neighbours is considered in the KNN model. Since the model tries to calculate the closest distance to several known points (labelled data), the accuracy of the data relies on what exists near that point and the number of valid preferences. It is harder to predict trends in fine-tuning the model. In this run, the neighbour of 1 achieves the highest accuracy, and the accuracy declines as the number of preferences or neighbours increases.

The results imply that the testing data has a strong local structure, meaning that individual data points are most accurately classified when relying on their nearest neighbours rather than broader patterns. The fact that a KNN model with $k = 1$ achieves the highest accuracy suggests that the data points are well-separated in their local regions, but increasing the number of neighbours introduces noise or ambiguity, leading to a decline in accuracy.

Additionally, the overlap between Robot 2 and Robot 4 when predicting Robot 4 suggests that these classes have some similarities or are not separable in the feature space, which could indicate either noisy data, feature limitations, or inherent similarities between the two robot types. The high specificity (99%) indicates that the model is particularly good at identifying negative cases, suggesting an imbalance in class distribution or that non-overlapping classes are distinctly different

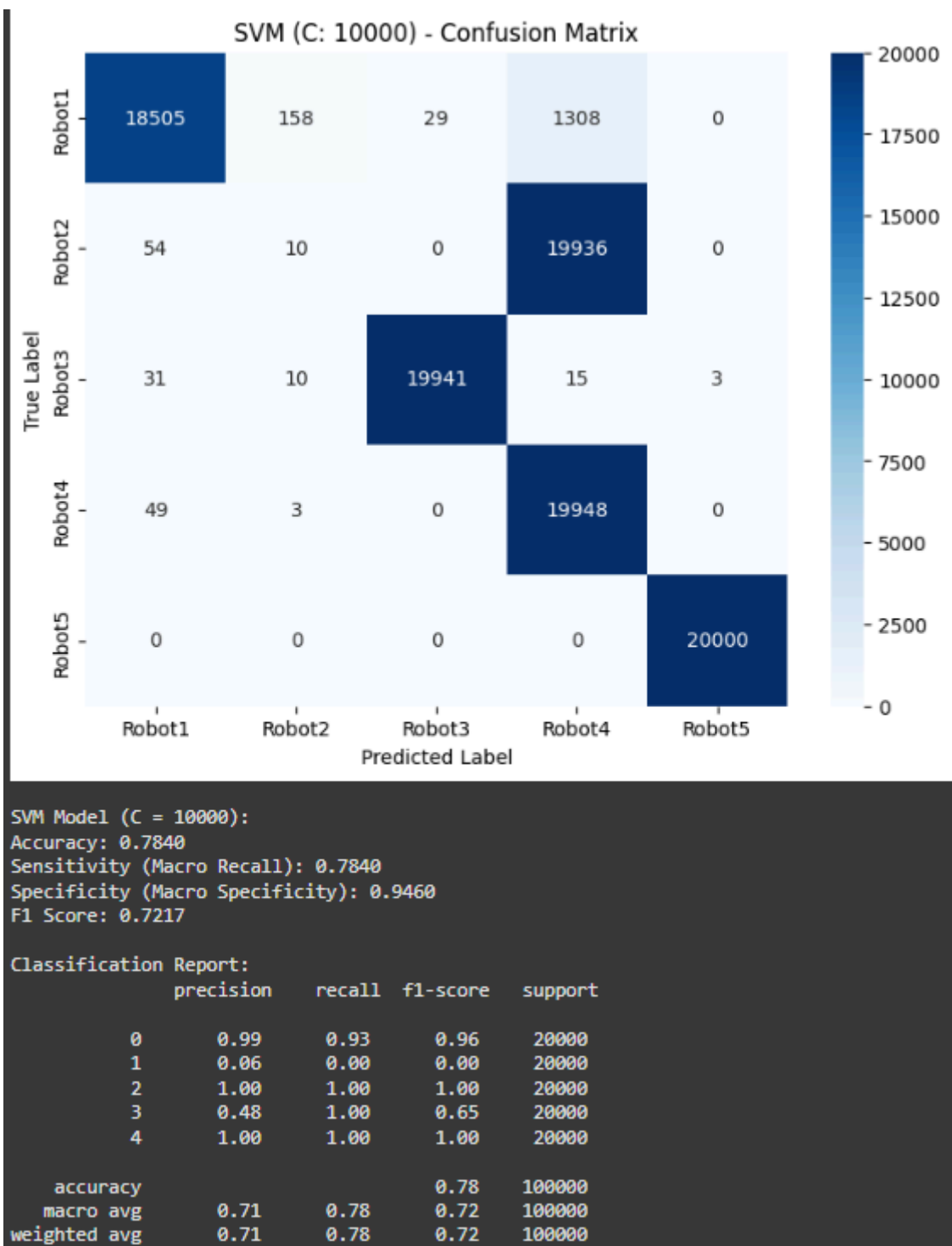


Figure: Support Vector Machine on the First Run

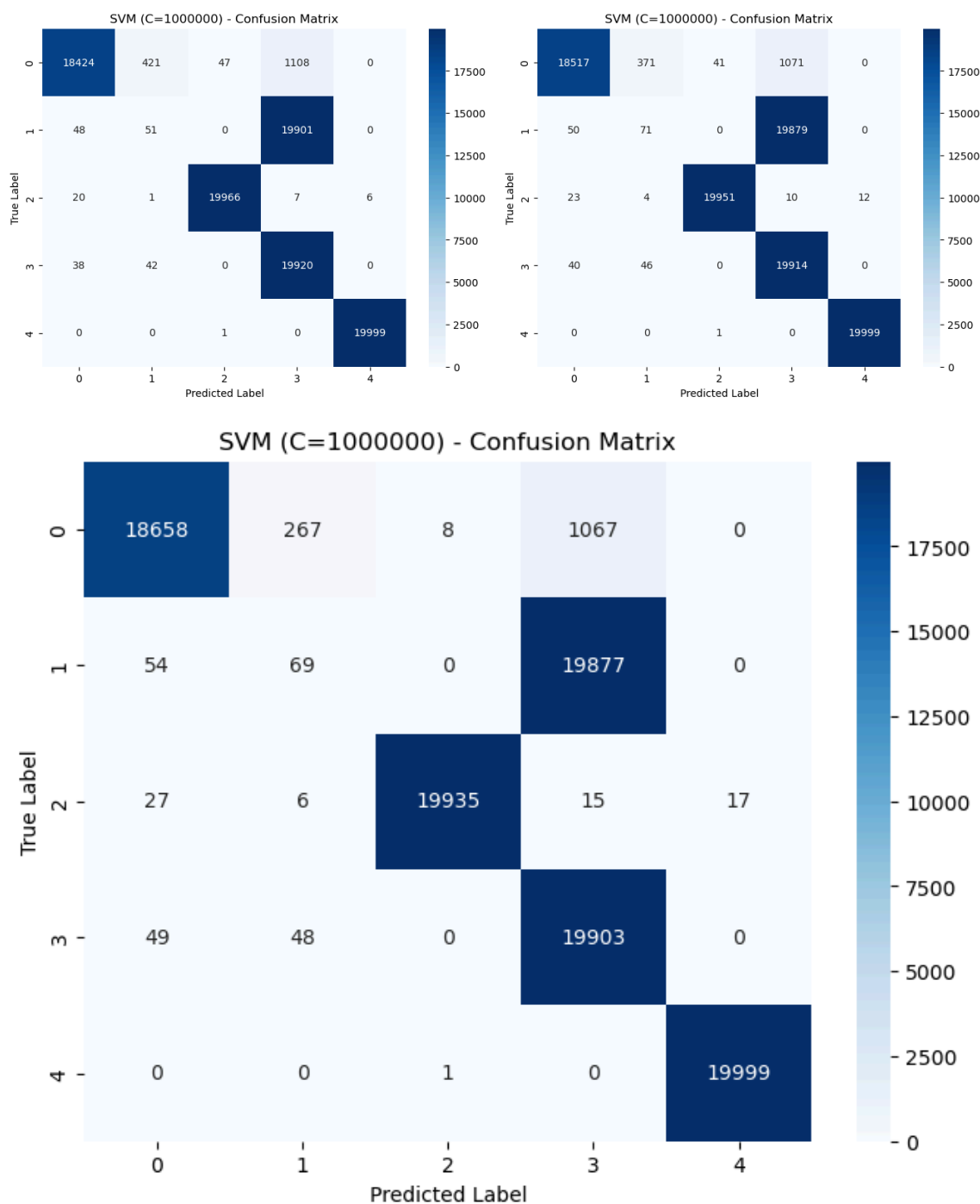


Figure: Performance of Chunking Dataset to SVM and Voting for Final Results. Chunk Size: 10, 20 and 40. Top to Bottom, Left to Right.

Chunk Number	Accuracy	Sensitivity	Specificity	F1 Score
10	0.7836	0.7836	0.955668	0.722579
20	0.78452	0.78542	0.955884	0.723457
40	0.78564	0.78564	0.956184	0.724111

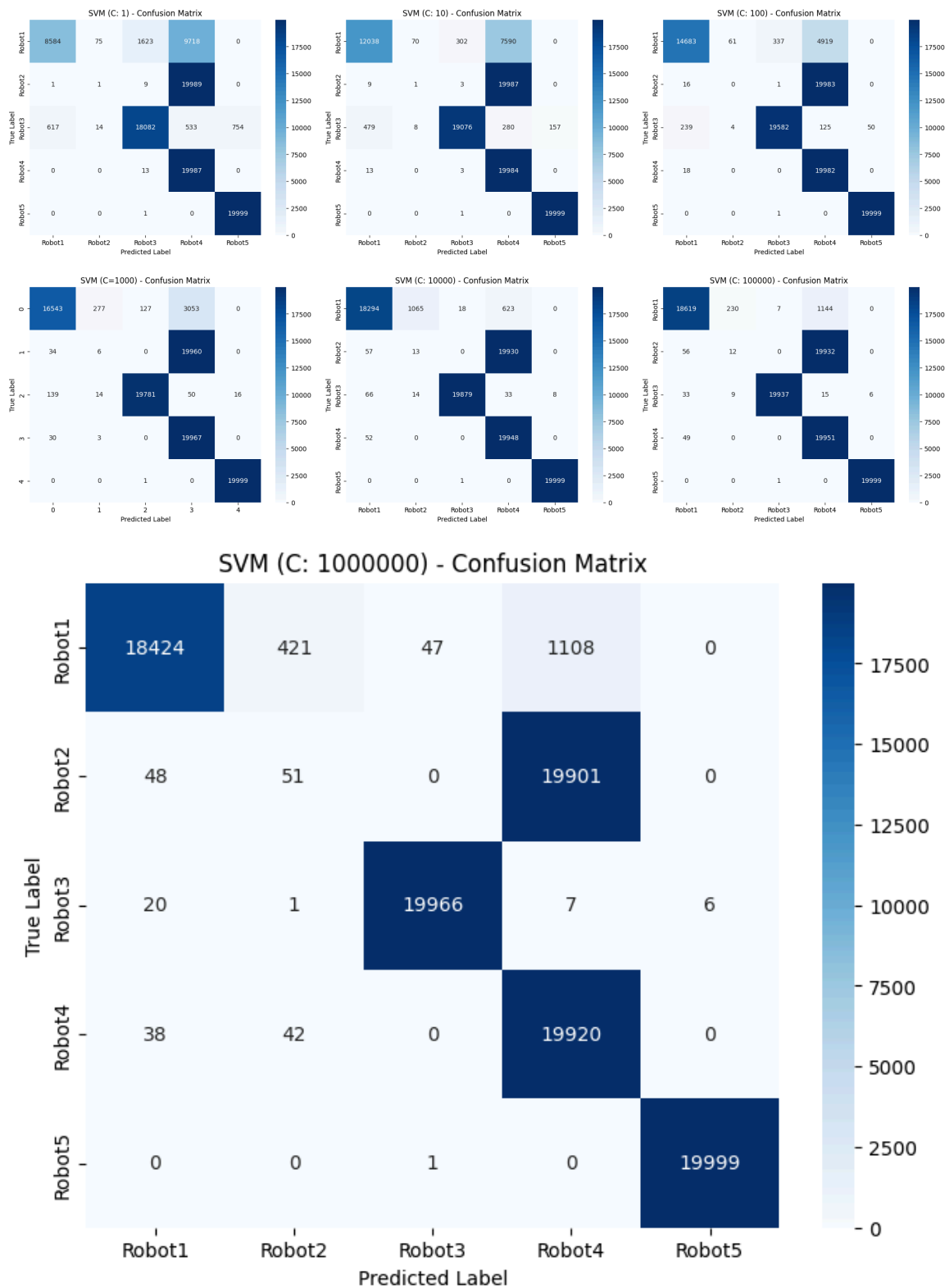


Figure: Performance of C Parameters in SVM Models' Accuracy, from 10^0 to 10^6

The SVM model is by far the most time-consuming model for training. The C parameter is unrealistic since it causes overfit to the data as there are not many missed predictions. Another problem with this model is that it has trouble when trying to classify Robot 4 because it got mixed results of Robot 4, Robot 2 and a bit of Robot 1. With the long training time and a similar problem to the decision tree, this is the least favourable model.

Take a look at the advantages and disadvantages of working with SVM. The SVM model works with small to medium datasets with well-separated classes (overlap in prediction for Robot4 with Robot 2), highly dimensional data (there are many features in this dataset), non-linearly separable problems, and requires high interpretability and control. The model does not perform well with large datasets (for this lab, the dataset may be too large), highly imbalanced data (Robot 2 has a high disparity in data), and fast prediction (it is extremely time-consuming to train and process the data)

With the effort of tuning different C parameters, there is a clear trend that the model can correctly predict Robot 1, Robot 3, and Robot 5, but completely miss Robot 2 and have confusion between Robot 4 and Robot 2 when predicting Robot 4. A higher C parameter makes this behaviour more obvious.

Chunking into smaller datasets for training might help speed up the training process, but it does not negate the nature of this dataset (not well-separated, imbalanced, and too large). The performance on the classification report is the same as fetching the whole dataset into the model.

Overall, the dataset and the model do not have a good match. The data seems to have some overlap in class boundaries and imbalanced data. The large dataset leads to time-consuming training sessions. The high C parameter causes overfitting, causing the model to memorize the data rather than generalize the pattern of data.

Comparison between models

In this experiment, the KNN model can predict accurately all the robots, plus follow the nature of this dataset. The random forest model has a rough guess about the existence of Robot 2 and Robot 4, but accurately predicts the rest of the robots. The decision tree model completely misses when predicting Robot 2, while mixing up the existence of Robot 2 and Robot 4 when predicting Robot 4. The support vector machine is by far the worst model for this dataset since it completely misses when predicting Robot 2, and has confusion in classifying Robot 4 into Robot 2 and a bit of Robot 1. The nature of this dataset is not a good fit for this model.

To be more specific, the KNN model with 1 neighbour has the highest accuracy (about 97%) for classification of all of the models introduced in this experiment. A decision tree with a maximum depth of 100 holds the highest accuracy in all decision tree models (about less than 78% score). The random forest model with 100 estimators and a maximum depth of 100 holds 98% accuracy on some experiment runs, but typically 81%. The SVM hold about 70% accuracy due to being a misfit to the dataset. The most accurate SVM model for this dataset is about 79% with 40 split chunks and a C parameter of 1,000,000

Pseudo Code Completion

```
from pathlib import Path
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import Normalizer
import pickle
import statistics
import pandas as pd
import math

Data = pd.read_csv(Path("/content/Classification of Robots from their
conversation sequence.csv"))
Data= Data.dropna()

train_set_Map, test_set= train_test_split(Data, test_size=0.2,
random_state=42)

Data_Map = train_set_Map.copy()
Data_test_R= test_set.drop(["Robot"], axis=1)
Data_labels_test_R= test_set["Robot"].copy()

chunk_number=10
chunk_size =math.ceil(len(Data_Map)/chunk_number)
data_chunked={}
m=0

for i in range(0, len(Data_Map), chunk_size):
    if m==chunk_number-1:
        chunk_size= len(Data_Map)- m *chunk_size
        data_chunked[str("chunk"+str(m))] = Data_Map[i:i+chunk_size]
        print(chunk_size)
        m=m+1

for i in range(0,chunk_number):
    Data_M = data_chunked[str("chunk"+str(i))].drop(["Robot"], axis=1)
    Data_labels_M = data_chunked[str("chunk"+str(i))]["Robot"].copy()
    SVMC = Pipeline(steps=[('Normal', Normalizer()), ('SVMC',
SVC(C=10000000, kernel='rbf'))])
    SVMC.fit(Data_M,Data_labels_M)
    pickle.dump(SVMC, open('SVMC.pkstr(i)', 'wb'))
    print(i)

OUT_SVMC={}
store = [[0 for col in range(chunk_number)] for row in
range(len(Data_test_R))]
```

```
Lables_SVMC_predicted=[[0 for col in range(1)] for row in  
range(len(Data_test_R))]
```

```
for i in range(0,chunk_number):  
    SVMC1 = pickle.load(open('SVMC.pkstr(i)', 'rb'))  
    OUT_SVMC[str("chunk"+str(i))]=SVMC1.predict(Data_test_R)  
    print(i)
```

```
for j in range(0,len(Data_test_R)):  
    for i in range(0,chunk_number):  
        store[j][i]=OUT_SVMC[str("chunk"+str(i))][j]  
    Lables_SVMC_predicted[j]=statistics.mode(store[j])
```